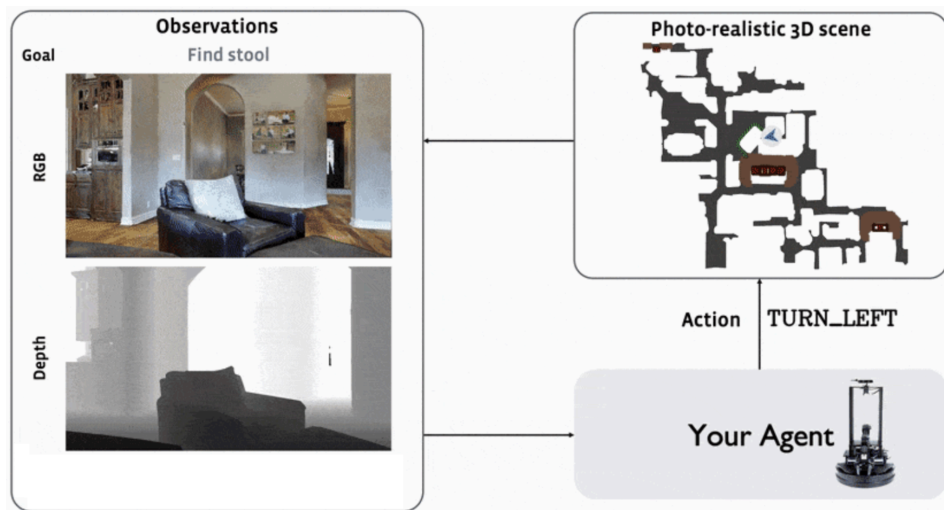


AI Capstone HW2

3D Scene Reconstruction

Announce: 3/26, Deadline: 4/9(Thursday) 23:59

Introduction



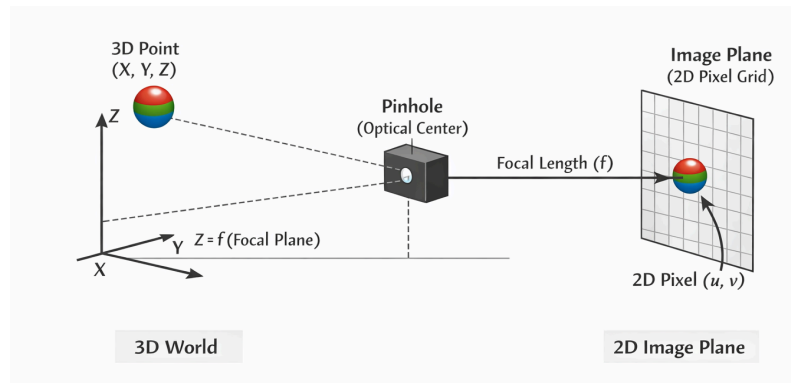
Habitat Navigation Challenge 2022 [1]

Imagine that you are building a **household robot** that can move around a home. Before it can perform tasks such as finding objects, cleaning rooms, or assisting people, it must first **understand the structure of its environment** so that it knows where things are and how to move safely and efficiently. For example, if you ask the robot to get something from the fridge, it needs to know where the kitchen is and how to navigate there without colliding with the dining table.

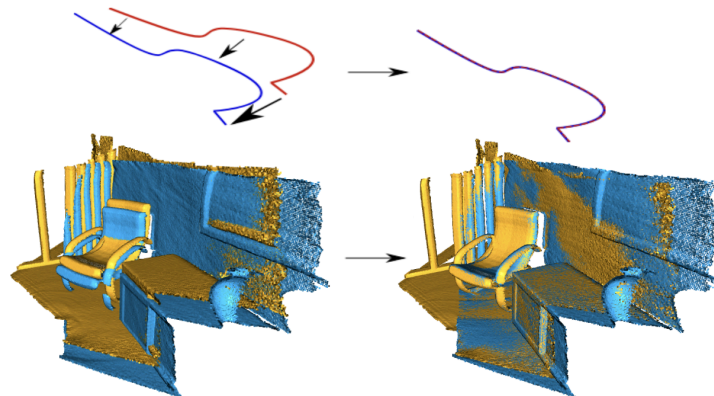
In this homework, you will take on the role of this robot and **reconstruct a 3D map of an apartment** from raw RGB-D data. You will operate an agent in the `apartment_0` scene from the Replica dataset [2] using Habitat-Sim, collecting observations, converting them into 3D point clouds, and aligning them into a consistent global scene. With this map, the robot can navigate the environment and perform everyday tasks. By completing this homework, you will gain hands-on experience in:

- **Data Collection:** You will control an agent to explore an environment and collect RGB-D data. How should the robot move to cover the entire space efficiently?

- **3D Reconstruction:** You will convert depth images into 3D point clouds to represent the scene. Through this process, you will gain an intuitive understanding of the **pinhole camera model** [3] and how depth values are projected into 3D space.



- **Scene Alignment:** You will align multiple point clouds with **Random Sample Consensus (RANSAC)** [4] and **Iterative closest point (ICP) algorithm** [5] to build a global map, combining observations from different viewpoints into a unified representation.



- **Evaluation:** You will evaluate the quality of your reconstruction by comparing estimated and ground truth trajectories, and analyze how different choices affect the final result.

While this assignment is conducted in a virtual environment, it serves as a starting point for considering the challenges involved in building and using such maps for real-world robotic tasks.

GitHub repository:

<https://github.com/HCIS-Lab/ai-capstone26/tree/main/hw2>

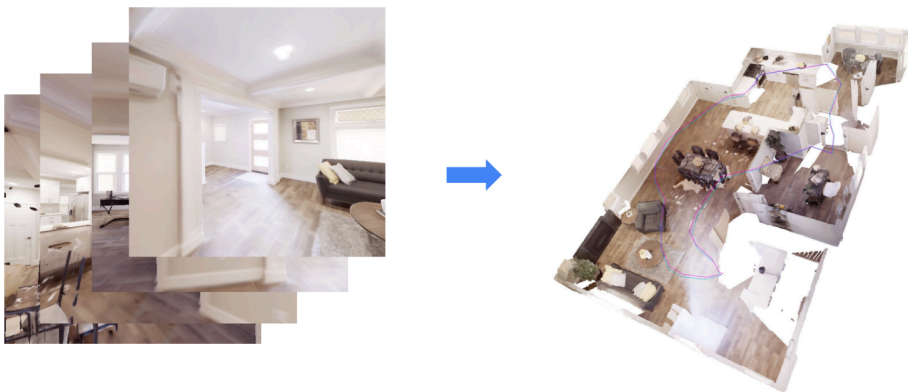
Implementation

We provide code templates: `load.py` for data collection and `reconstruct.py` for implementing reconstruction algorithms, please check [hw2 readme](#) on github. You will need to complete the sections marked with #TODO.

Task 1: Data Collection

- **Dataset Download:** Download the `apartment_0` environment from the [Replica Dataset](#).
- **Data Collection:** Control the agent to explore the **first floor and second floor of apartment_0**. Save the RGB images, depth images, and the camera ground truth poses as `GT_pose.npy`.

Task 2: ICP Alignment & 3D Map Reconstruction



- **Pipeline:** Unproject depth images → Voxelization (Downsampling) → Global Registration (RANSAC) → Local Registration (ICP).
 1. **Unprojection:** Convert depth images into 3D point clouds.
 - Camera Model: Pinhole (Resolution 512x512).
 - FOV: Horizontal and Vertical are both 90 degrees.
 - Depth_scale = 1000
 - *Note: You should **not use Open3D functions here**; instead, you should implement each step yourself.*
 2. **Voxelization (Downsampling):** Reduce the number of points to improve computational efficiency.
 3. **Global Registration (RANSAC):** Estimate an initial alignment between point clouds to handle large viewpoint differences.

4. **Local Registration (ICP):** Refine the alignment by minimizing the distance between corresponding points.

- **Required and Bonus Tasks:**

1. **Open3D Version (Required):** Use Open3D's built-in registration function (RANSAC, voxel down, Point-to-Point/Plane ICP) [6].
2. **My ICP Version (Bonus):** Implement your own ICP algorithm, including nearest neighbor search and transformation estimation.

*Note: Your own ICP should **complete within 5 minutes during the demo**; otherwise, you will not receive the bonus points.*

Task 3: Camera Pose Estimation & Visualization

- **Scene:** Accumulate point clouds into a 3D scene and **remove the ceiling** for better observation.
- **Trajectory:** Visualize both the **Estimated Trajectory (Red)** and the **Ground Truth Trajectory (Black)**.
- **Evaluation:** Calculate the **Mean L2 Distance** between estimated camera positions (x,y,z) and ground truth positions (x,y,z).

Example results

Red line: The estimated camera pose

Black line: The ground truth camera pose (GT_pose.npy)

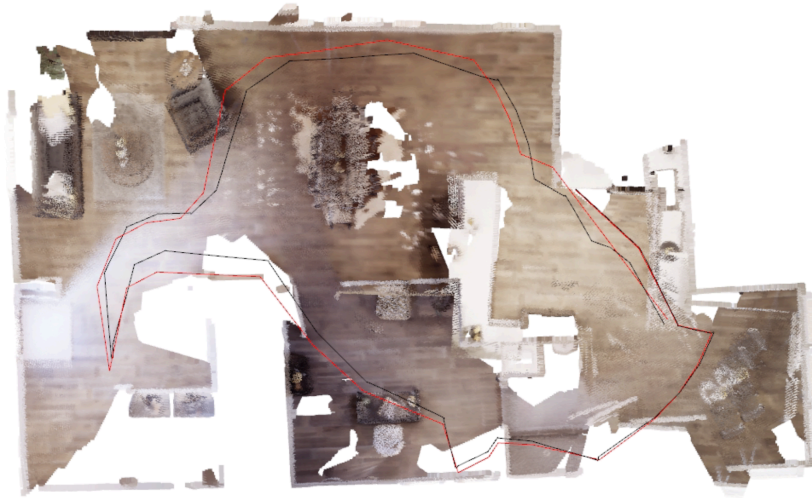
Note: the below results are the screenshots.

- **The first floor of apartment_0**

Data size = 152



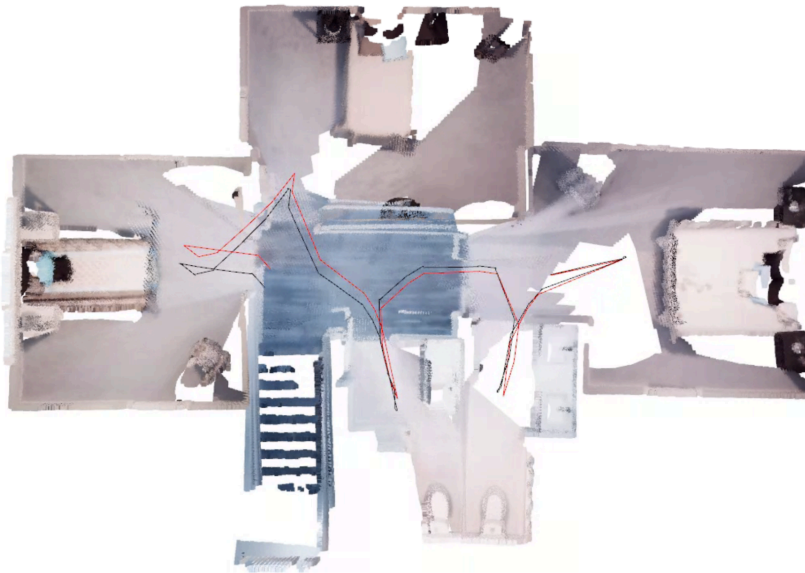
Data size = 129



The above two results show the **importance of the data collection step**. Although collecting data in the same scene, with different data size, trajectory, distance to the object, the reconstructed result differs a lot.

- **The second floor of apartment_0**

Data size = 213



Environment Setup & Troubleshooting (Linux & macOS)

While many systems run the provided code without issues, certain hardware and library combinations, particularly **macOS with Apple Silicon (M1-M4)**, may

encounter environment-related crashes. If you experience unexpected "Segmentation Faults" or crashes during execution, please follow these troubleshooting steps :

1. The NumPy 2.0 Compatibility Issue

Currently, Open3D (v0.16 - v0.18) may not be fully compatible with NumPy.

- a. **Symptom:** A Segmentation Fault occurs immediately when converting arrays to Vector3dVector.
- b. **Potential Fix:** Downgrade to **NumPy 1.26.4**.
- c. **Command:** `pip install numpy==1.26.4`.

2. Potential Library Conflicts (GLFW)

On some systems, importing `habitat_sim` and `open3d` in the same script leads to a **GLFW Library Conflict**, causing a crash during window initialization.

- a. **Recommended Fix:** If you encounter crashes during visualization, we suggest using **separate Conda environments**:
 - i. **Env A (Data Collection):** Use your habitat environment for `load.py`.
 - ii. **Env B (Reconstruction):** Use a clean `open3d` environment for `reconstruct.py`.

3. Missing System Dependencies

Ensure your operating system has the necessary multi-threading libraries installed:

- a. **macOS:** If Open3D fails to load, try installing `libomp` via Homebrew: `brew install libomp`.
- b. **Linux:** If you encounter an `ImportError` related to OpenMP, run: `sudo apt-get install libomp-dev`.

Grading

Online Demo (30%)

After the deadline, please sign up for an online demo slot. You can choose one of the following tracks based on your implementation:

- **Standard Track:** For students who implemented only the Open3D ICP.
- **Bonus Track:** For students who implemented Open3d ICP and their own ICP version. A longer demo time will be provided for this track.

Note:

- The **standard track demo will last 5 minutes**, and the **bonus track demo will last 7 minutes**.
- During the demo, you should present your reconstruction results on your computer, including: the reconstructed 3D scene (with the ceiling removed), the estimated and ground truth trajectories (Red/Black), and the Mean L2 error, as examples given above. Also, we will ask you questions about implementation, and your score may be adjusted based on how clearly you explain your approach.
- You may freely decide which track to join. However, if you wish to receive the 20% Bonus points, you **MUST** choose the Bonus track. Failure to demonstrate the custom version will result in no bonus points being awarded, even if the code is submitted.

Report (70%)

Your report should be **written in English**, be concise, and include the following content:

- **Implementation (50%)**
 - **Code:**
 - Provide a clear explanation of how you implement each step of the pipeline.
 - **Result & Discussion:**
 - Screenshots of apartment_0 Floor 1 reconstruction.
 - A comparison table that includes:
 - Mean L2 distance between estimated and ground truth camera poses.
 - Total execution time.
 - The hyperparameters you have tested.

- Discuss the results and share your findings.
 - (Bonus) If you implement your own ICP algorithm, compare your results with the Open3D version and highlight the techniques or tricks you used to improve performance.
- **Questions (20%):**
 - What happens if you perform ICP without Global Registration (RANSAC)? Why?
 - Describe any tricks used to improve your ICP stability (e.g., Voxel size, Huber loss, etc.).

Bonus (+20%)

Custom own ICP: Successful implementation of **my_local_icp_algorithm** with performance comparable to the Open3D version.

Q&A Page

If you have any questions about this homework, please ask them on the following Notion page. We will answer them as soon as possible. Additionally, we encourage you to answer other students' questions if you can.

<https://www.notion.so/Homework-2-Q-A-Page-312c9908e88680f782b8f0aebacf439e>

Submission

Due Date: 2026/04/09 23:59

Please directory compress all your code files and report (.pdf) into

{STUDENT ID}_hw2.zip and submit it to the New E3 System.

ex : 123456789_hw2.zip

The file structure should look like:

```

{student_id}_hw2.zip
├─ 📄 load.py          # Only for exploring the scene and save the data
├─ 📄 reconstruct.py   # Main implementation (Open3D + My ICP).

```

└─ 📄 report.pdf	# Detailed report
└─ 📄 README.md	# Instructions on how to set up the environment and run the code.

- **Wrong submission format leads to -10 points.**
- **Late submission leads to -20 points per day.**
- **Plagiarism is strictly prohibited, including assignments written on GitHub. If we find any instances, it will result in a zero score without any room for objection.**
- **Reports not written in English will receive a score of zero.**

References

- [1] [Habitat Challenge 2022](#)
- [2] [The Replica Dataset: A Digital Replica of Indoor Spaces](#)
- [3] [Multiple View Geometry in Computer Vision. Cambridge University Press.](#)
- [4] [Understanding Iterative Closest Point \(ICP\) Algorithm with Code](#)
- [5] [A Comparative Analysis of RANSAC Techniques Leading to Adaptive Real-Time Random Sample Consensus](#)
- [6] https://www.open3d.org/docs/0.19.0/tutorial/pipelines/global_registration.html